

**Computer
Vision**
AI-4002

Lab 03

Geometric Transformation

National University of Computer & Emerging Sciences –
NUCES – Karachi



National University of Computer & Emerging Sciences -
NUCES - Karachi
Department of Artificial Intelligence

Course Code: AI-4002	Computer Vision Lab
-----------------------------	----------------------------

1. Objectives.....	2
2. What is Geometric Transformation	2
3. The Image Coordinate System	4
4. 2D Transformation vs 3D Transformation	6
4.1 2D transformation.....	6
4.2 3D transformation.....	7
5. Transformation Matrix.....	8
5.1 The Anatomy of a 2x2 Transformation Matrix	8
5.2 Reading the Vectors	9
5.3 What Each Variable Controls.....	9
6. 2D Geometric Transformations.....	10
6.1 Affine Transformation	10
6.1.1 Linear Transformation.....	12
6.1.1.1 The Mathematical Foundation.....	12
6.1.1.2 What Linear Transformations Can Achieve	13
6.1.1.3 The Limitation: The Anchored Origin	15
6.1.1.4 The solution: Homogenous Coordinates.....	16
6.2 Projective Transformations.....	17
6.2.1 The Mathematical Core	18
6.2.2 The Denominator Effect	18
6.2.3 Modeling Real-World Perspective	18



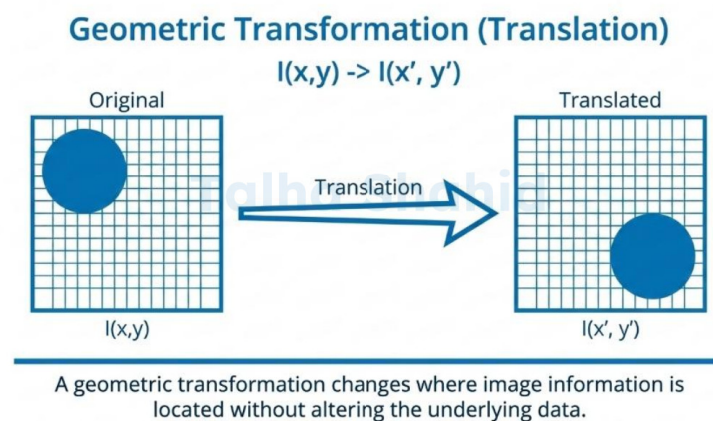
1. Objectives

1. Explain what a geometric transformation is.
2. Distinguish geometric and photometric transformations.
3. Explain the image coordinate system used in computer vision.
4. Distinguish linear, affine and projective transformations.
5. Explain homogeneous coordinates.

2. What is Geometric Transformation

"A geometric transformation is a mathematical operation that alters the spatial position, orientation, scale, or shape of an object, without changing its underlying data values".

In simple words, a geometric transformation changes where image information is located.



An image can be viewed as a function

$$I(x,y)$$

where:

- x = horizontal image coordinate
- y = vertical image coordinate
- $I(x,y)$ = pixel intensity/color at that location
- $I(x',y')$ = transformed image



National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

Coordinate Manipulation: Unlike *photometric transformations* (which change a pixel's color, brightness, or contrast), geometric transformations change *where the pixel is located, not what the pixel looks like*.

Geometric transformation:

$$(x, y) \rightarrow (x', y')$$

Photometric transformation:

$$I(x, y) \rightarrow I'(x', y')$$

Examples of geometric transformations:

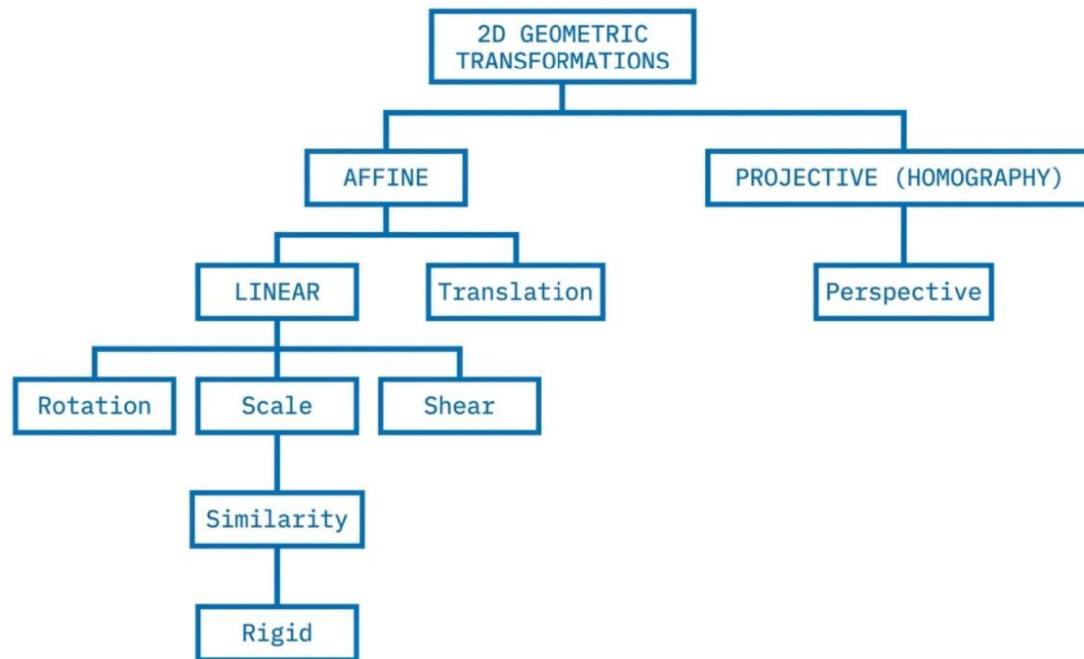
- affine transformation
 - linear transformation
 - rotation
 - scaling
 - shearing
- perspective transformation
 - translation

Examples of photometric transformations:

- Point Transformation
 - brightness adjustment
 - contrast enhancement
 - gamma correction
 - color conversion

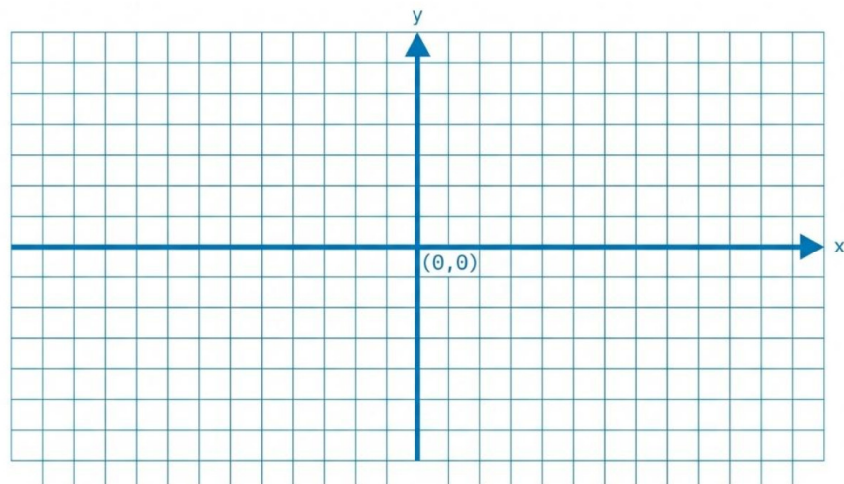
This distinction is important:

Geometry changes where the pixel is. Photometry changes what the pixel looks like.



3. The Image Coordinate System

In mathematical Cartesian coordinates we usually imagine:

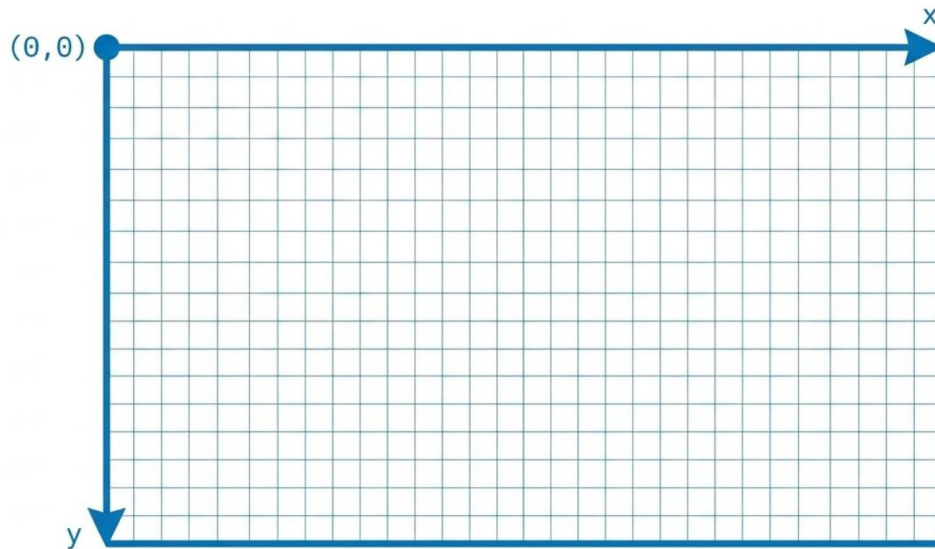




National University of Computer & Emerging Sciences - NUCES - Karachi

Department of Artificial Intelligence

However, images normally use:



Therefore:

$x \rightarrow$ right

$y \rightarrow$ down

Talha Shahid

The origin is normally the **top-left corner**.

OpenCV also represents image points using this image-coordinate convention. In its rotation documentation, OpenCV explicitly accounts for the top-left-origin coordinate system when defining positive rotation.

Example

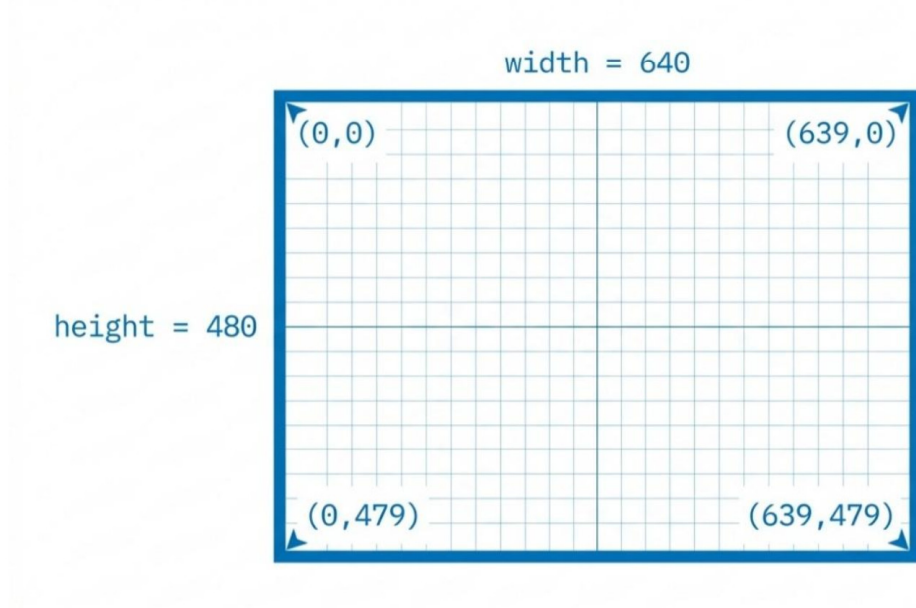
Suppose an image has:

width = 640

height = 480



Then approximately:



Note that:

`image[y, x]`

but a geometric point is commonly written as:

(x,y)

4. 2D Transformation vs 3D Transformation

4.1 2D transformation

A 2D transformation operates on points:

$$(x,y) \rightarrow (x',y')$$

It is appropriate when dealing with:

- an image plane
- a photograph
- a document
- a planar object

Typical matrix representations are:

$$2 \times 3$$



for affine warping, or

$$3 \times 3$$

using homogeneous coordinates.

OpenCV's `warpAffine()` accepts a 2×3 matrix, while `warpPerspective()` uses a 3×3 matrix.

4.2 3D transformation

A 3D transformation works with:

$$(x, y, z) \rightarrow (x', y', z')$$

Typical homogeneous representation:

$$4 \times 4$$

For example, robotics and 3D graphics frequently use:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = T \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

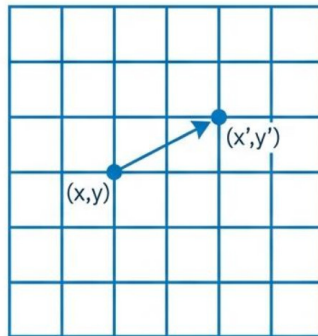
Computer vision often eventually connects 3D world coordinates to 2D image coordinates:

$$(X, Y, Z) \rightarrow (x, y)$$

That is a separate topic involving camera projection and calibration.



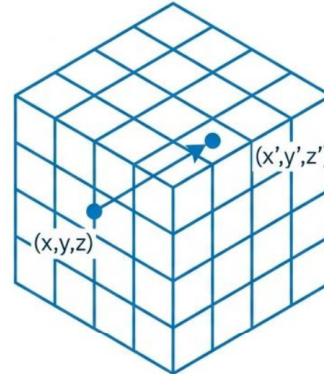
2D TRANSFORMATION



Operates on: Image planes,
Photos, Documents

Representation: 2x3 Affine
or 3x3 Homogeneous

3D TRANSFORMATION



Operates on:
3D World Space

Representation:
4x4 Homogeneous

Talha Shahid

5. Transformation Matrix

A transformation matrix is a mathematical map that encodes exactly how to warp, rotate, scale, or shear a 2D coordinate space. When you multiply an original coordinate vector by this matrix, it calculates the new, transformed location of that point.

5.1 The Anatomy of a 2x2 Transformation Matrix

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

The transformation matrix is composed of two distinct vectors, but to visualize how it alters space, you must read the matrix vertically as **columns**, rather than horizontally as rows. The two column vectors tell you exactly where the foundational axes of the entire grid end up after the transformation.



5.2 Reading the Vectors

Think of the original coordinate grid as being built from two "basis vectors": the unit X-axis at (1, 0) and the unit Y-axis at (0, 1). The matrix simply tells you their new coordinates.

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- **The First Column Vector:** This is the new landing spot for the X-axis.

$$\begin{bmatrix} a \\ c \end{bmatrix}$$

- **The Second Column Vector:** This is the new landing spot for the Y-axis.

$$\begin{bmatrix} b \\ d \end{bmatrix}$$

Every other point on the grid just goes along for the ride, automatically redrawing itself based on where those two primary axes landed.

5.3 What Each Variable Controls

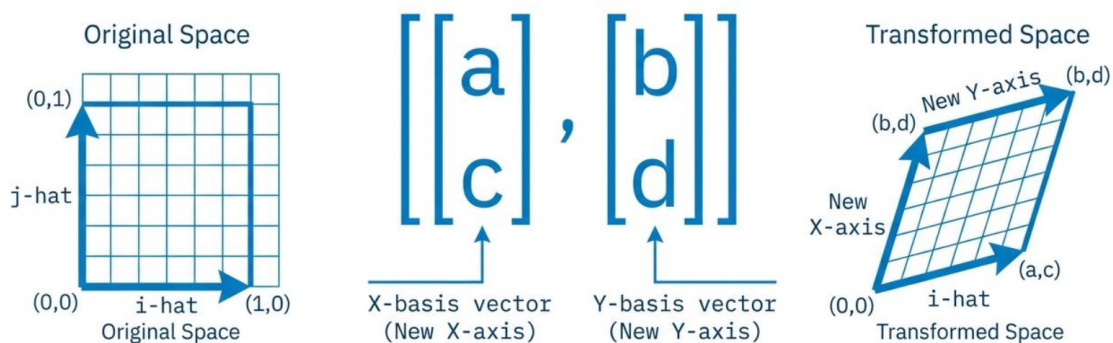
By manipulating the individual values of \$a\$, \$b\$, \$c\$, and \$d\$, you control specific geometric behaviors:

Variable	Structural Role	Visual Effect
a	X-coordinate of the new X-axis	Horizontal Scaling. Values > 1 stretch the grid horizontally. A negative value flips the grid horizontally.
c	Y-coordinate of the new X-axis	Vertical Shearing. Changes this value tilt the vertical grid lines up or down, breaking perpendicularity.
b	X-coordinate of the new Y-axis	Horizontal Shearing. Changes this value tilt the horizontal grid lines left or right, creating the shear effect seen in your previous diagram.

d	Y-coordinate of the new Y-axis	Vertical Scaling. Values > 1 stretch the grid vertically. A negative value flips the grid vertically.
----------	--------------------------------	---

If you want a pure rotation, you cannot just change one variable. You must change all four simultaneously using trigonometric functions (sine and cosine) so that both axes tilt and scale perfectly in unison to maintain their 90-degree relationship.

Anatomy of a 2x2 Transformation Matrix



a	Horizontal Scale (X of X-axis)
c	Vertical Shear (Y of X-axis)
b	Horizontal Shear (X of Y-axis)
d	Vertical Scale (Y of Y-axis)

6. 2D Geometric Transformations

6.1 Affine Transformation

An **affine transformation** is a fundamental geometric mapping that preserves collinearity (points on a line remain on a line) and parallelism (parallel lines remain parallel). While it can alter angles and distances, it always preserves the ratio of distances between points lying on a straight line.

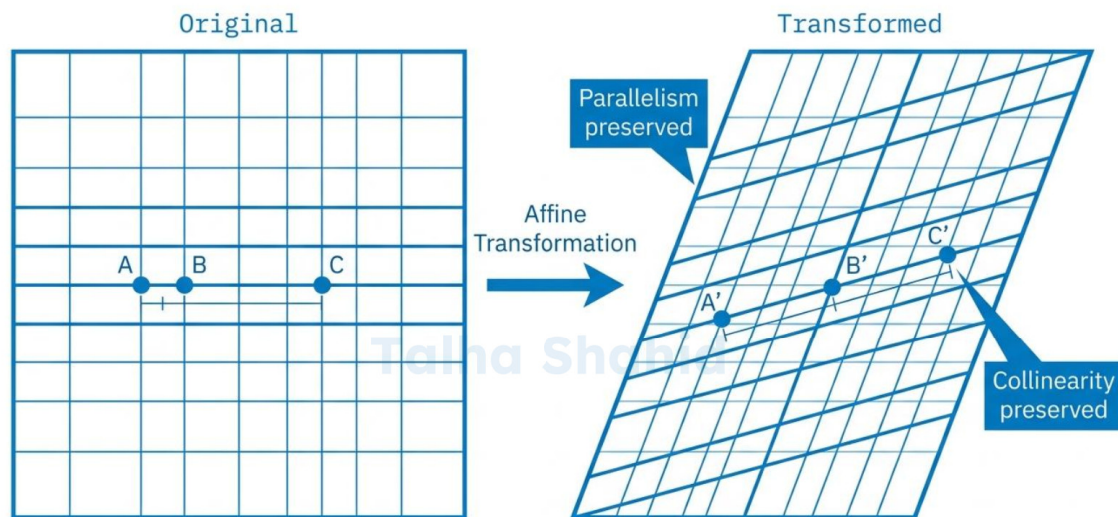
Mathematically, an affine transformation is simply the composition of a linear transformation and a translation. It is expressed by the equation:

$$p' = Ap + b$$

$$p' = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

where:

- A represents the linear transformation matrix,
- x is the original coordinate vector, and
- b is the translation vector



Note: This diagram illustrates the core geometric properties of an affine transformation by comparing a standard 2D coordinate space to its transformed state (warped Parallelogram).

The Original Coordinate Space (Left)

- The grid represents a standard Cartesian plane composed of perpendicular intersecting lines and perfect squares.
- Points A, B, and C are plotted along a single horizontal axis.
- A specific, measurable distance ratio exists between the points (e.g., the length of segment AB compared to the length of segment BC).

The Transformed Space (Right)

- The grid demonstrates the resulting coordinate space after an affine mapping—specifically a shear operation—is applied.



- The uniform right angles of the original grid are skewed, warping the coordinate system and stretching the original squares into tilted parallelograms.

Core Properties Preserved Despite the structural distortion of the grid, the affine transformation strictly enforces two fundamental mathematical rules:

- **Collinearity:** Points A', B', and C' remain positioned on a perfectly straight, continuous line. While the line itself has changed its angle, it does not curve, bend, or break. Furthermore, the ratio of the distances between these points (A'B' relative to B'C') remains mathematically identical to the original ratio of AB to BC.
- **Parallelism:** The grid lines that were parallel in the original space stretch and tilt uniformly together. Lines that never intersected in the original Cartesian grid remain strictly parallel and will never intersect in the transformed coordinate space.

6.1.1 Linear Transformation

A linear transformation is the foundational subset of an affine transformation that relies strictly on matrix multiplication, meaning it can warp, scale, and rotate space, but it is mathematically tethered to the origin point (0,0).

Note: A linear transformation is affine transformation without a translation vector.

6.1.1.1 The Mathematical Foundation

A linear transformation operates by multiplying a coordinate vector p by a 2×2 transformation matrix A :

$$p' = Ap$$

Expanded into its full matrix form, it looks like this:



$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

This multiplication dictates the new coordinates through two algebraic equations:

- $x' = ax + by$
- $y' = cx + dy$

6.1.1.2 What Linear Transformations Can Achieve

By altering the values of the matrix, this single mathematical operation can produce distinct geometric effects:

- **Scaling:** Resizes the object by multiplying coordinates by scale factors s_x and s_y . The diagonal placement in the matrix

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

ensures the axes scale independently without mixing.

- **Rotation:** Tilts the coordinate space around the origin. It uses

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

to perfectly balance both axes, ensuring they remain perpendicular as they rotate.

- **Reflection:** Flips the geometry like a mirror. A negative value on the diagonal will invert that axis (e.g.,

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

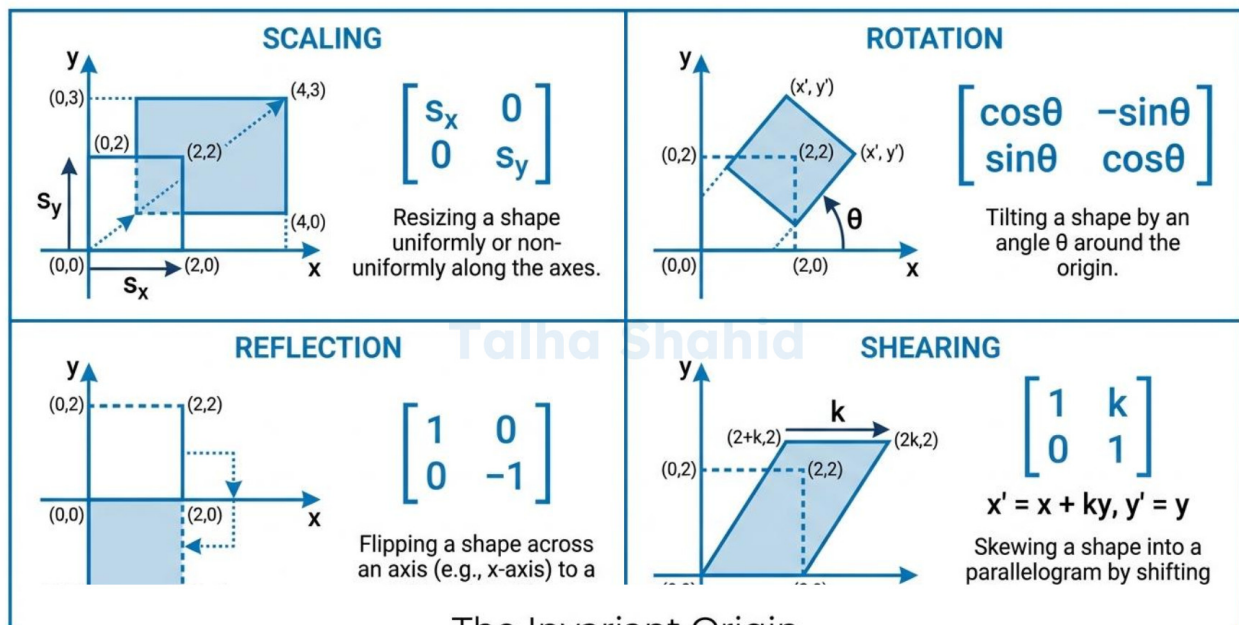
inverts the Y-values, reflecting the object across the X-axis).

- **Shearing:** Skews the grid by adding a fraction of one coordinate to another using a constant k . A horizontal shear matrix

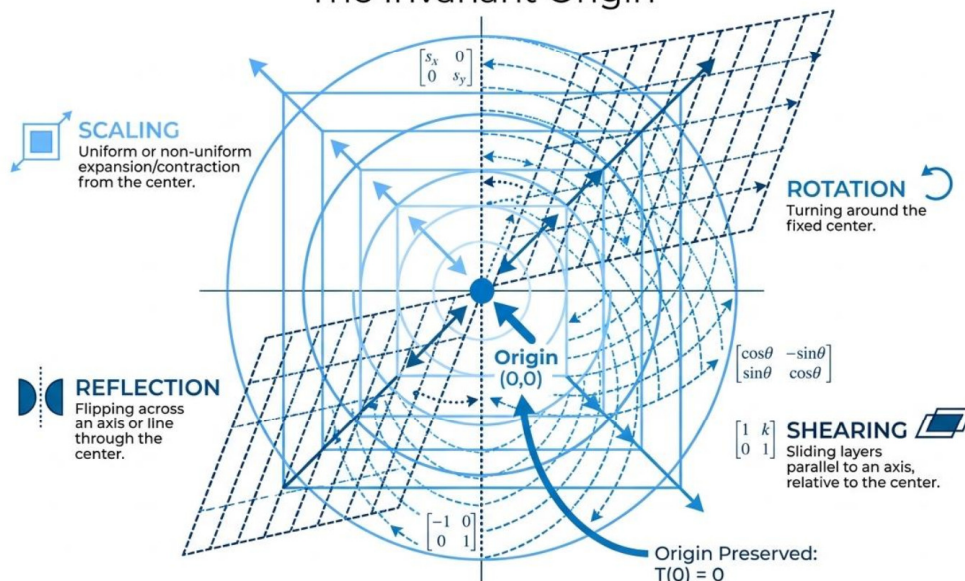
$$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

results in the equation $x' = x + ky$.

What Linear Transformations Can Achieve



The Invariant Origin



6.1.1.3 The Limitation: The Anchored Origin

This raises an important analytical question:

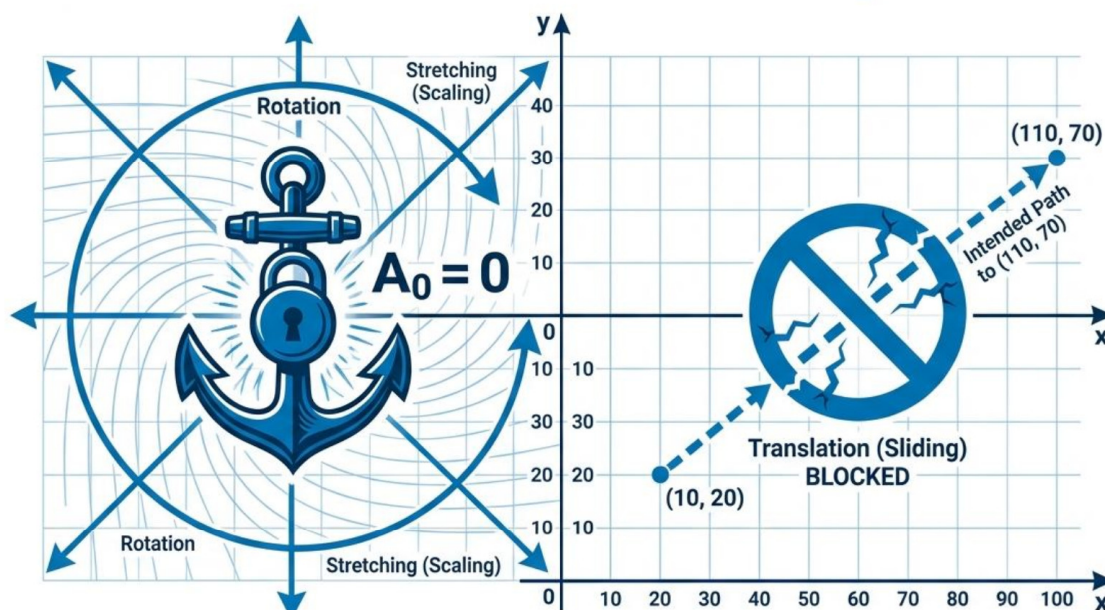
Can this matrix translate (slide) a point, such as moving (10, 20) to (110, 70)?

The mathematical answer is no. A strict rule of linear transformations is that any matrix multiplied by a zero vector results in a zero vector:

$$A\mathbf{0} = \mathbf{0}$$

If you multiply the origin (0,0) by any 2 x 2 matrix, the result will always be (0,0). Because the origin is permanently anchored, standard linear transformations cannot translate geometry. They can spin, stretch, and shear the grid, but they cannot slide it across the board. This specific mathematical limitation is what forces computer graphics systems to introduce a workaround to achieve full affine transformations.

The Limitation: The Anchored Origin



While the grid can spin (rotation) or stretch (scaling) around the center, the origin (0,0) is permanently fixed, preventing any sliding (translation) motion. The entire coordinate system is anchored at this single point.



6.1.1.4 The solution: Homogenous Coordinates

Homogeneous coordinates bypass the origin limitation by adding a temporary, dummy "1" to standard 2D coordinates, turning (x, y) into (x, y, 1). This mathematical trick allows computers to perform translations using only matrix multiplication, completely eliminating the need for a separate vector addition step.

The Problem with Vector Addition

The standard affine equation is

$$p' = Ap + b$$

This forces the computer to perform a matrix multiplication (Ap) for scaling or rotating, stop that process, and then perform a separate vector addition (+b) to slide the object. Graphics processors (GPUs) are deeply optimized to execute continuous, uninterrupted chains of matrix multiplications. Breaking that hardware pipeline to switch to vector addition creates a massive computational bottleneck when calculating millions of coordinate points per second.

The Matrix Multiplication Solution

By upgrading the math to a 3 x 3 matrix, we can fold the translation values t_x and t_y directly into the rightmost column:

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

When the computer executes this multiplication, the row-by-column math forces the "1" in the coordinate vector to multiply against the t_x and t_y in the matrix. These cleanly outputs the exact translation equations algebraically:

- $x' = (1)x + (0)y + (t_x)1 = x + t_x$
- $y' = (0)x + (1)y + (t_y)1 = y + t_y$



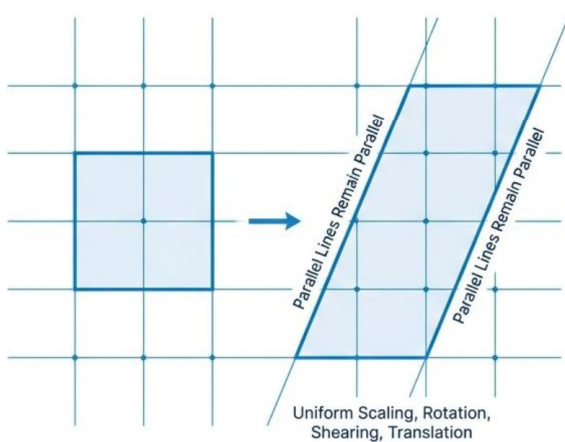
The Power of Composition

Because translation, rotation, and scaling are now *all* formatted uniformly as 3×3 matrix multiplications, they speak the exact same mathematical language. A computer can multiply a scale matrix, a rotation matrix, and a translation matrix together into one single "master matrix" *before* touching the geometry. The graphics engine then only has to perform one single matrix multiplication per pixel to apply all three transformations simultaneously.

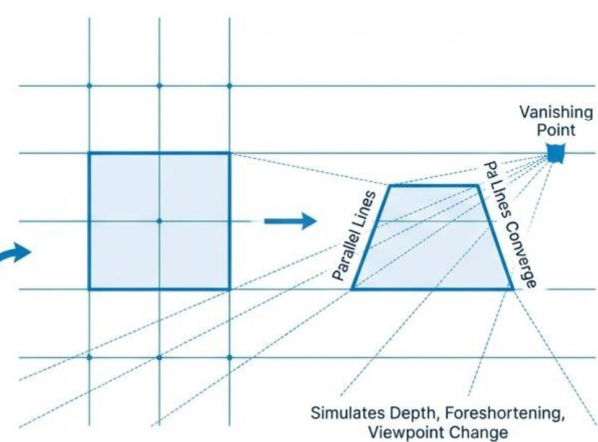
6.2 Projective Transformations

Projective transformations, or homographies, are a broader category of geometric mapping that break the strict rules of affine transformations to simulate how human eyes and cameras perceive 3D depth in a 2D space. While affine transformations always preserve parallelism, projective transformations allow parallel lines to converge, creating the illusion of perspective.

Affine: Preserves Parallelism



Projective: Simulates 3D Perspective



Core Difference:
Parallelism



6.2.1 The Mathematical Core

A projective transformation uses a full 3 x 3 transformation matrix, commonly denoted as H:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

To find the final 2D coordinates on a screen, the resulting vector must be normalized by dividing by the third scaling coordinate, \$w'\$. This yields the following algebraic equations:

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

6.2.2 The Denominator Effect

The denominator $h_{31}x + h_{32}y + h_{33}$ is the critical mathematical distinction between affine and projective geometry.

In an affine transformation, the bottom row of the matrix is strictly locked to $[0, 0, 1]$. Therefore, the denominator is always exactly 1, causing uniform scaling across the entire image.

In a projective transformation, the denominator fluctuates depending on the x and y values of the specific point being processed. As points "move further away" in the virtual space, the denominator grows larger, causing the resulting x' and y' coordinates to shrink closer together. This variable division is the mathematical engine that generates depth.

6.2.3 Modeling Real-World Perspective

Because of this dynamic denominator, projective transformations can accurately model how flat planes distort when viewed at an angle:

- **Converging Lines:** The visual effect of a straight road where the left and right parallel edges appear to pinch together at the horizon.
- **Camera Viewpoints:** How an object's perceived shape warps as a camera pans or tilts around it in 3D space.
- **Planar Alignment:** Photographing a rectangular document resting on a desk from an angle. The camera captures it as a trapezoid (narrower further away, wider up close). A projective transformation matrix can perfectly reverse this distortion, stretching the trapezoid back into a flat, properly proportioned rectangle.

